SE | SAE | Operator | QKD / KME | Peer / origin SAE (RMI) | End without keys

synchronize() eligibility (window 5 min)
INSERTED: no row or retention expired; stale RECORD_SYNCHRONIZATION, RECORD_FETCHING_STARTED or RECORD_SHARED; RECORD_SYNCHRONIZATION of the same issuer, or the issuer ID is higher than this SAE ID
ORIGIN_PROVISIONED: finished row, same target SAE
RECORD_SHARED_WITH_DIFFERENT_SAE: RECORD_SHARED, ORIGIN_PROVISIONED or a live RECORD_SYNCHRONIZATION for a different target SAE
NOT_INSERTED: fresh RECORD_FETCHING_STARTED; for the same target SAE a fresh RECORD_SHARED or a live RECORD_SYNCHRONIZATION of another issuer
Lifecycle: RECORD_SYNCHRONIZATION → RECORD_FETCHING_STARTED → RECORD_SHARED → ORIGIN_PROVISIONED
RMI: every call is mTLS with a pinned peer SPKI; issuingSaeId must be the caller; removeRecord / markOriginProvisioned only by a party; deleteShare by a non-party only for a completed share (handover)

PROVISION {unix}
PIN stays off USB

se_ready_provision()?
owner, device cert + key,
SAE CA, ML-KEM key;
time synced

No

Refused: TLS is not armed

Yes

TLS 1.3 mTLS, PURE_PQC
SE: SAE leaf vs. SAE CA
SAE: device leaf vs. client_ca

Uplink LV v5: session_sig, ecc_pub, client_hash,
slot_size, pad_count, fill_id, ML-KEM pk,
peers (hash, name, pad_plain)

in parallel

Erase QKD R-MEM slots
then send TROPIC_WIPE_FINISHED

SeSessionUplink.readAndVerify
client_hash = SHA-384(TLS SPKI ‖ ecc_pub)?
ECDSA over SHA-384(client_hash ‖ exporter)?

No

Abort (bad uplink)

Yes

Operator: choose peer SAE and peer device
(InputHandler.selectForUplink)
pad_plain = min(plainMax, peer pad_plain)

No: tryDelete, Reselect

Local synchronize()
(AtomicRecordStateMap)

INSERTED

NOT_INSERTED

NOT_INSERTED: Record sharing is in
progress on another SAE. Wait 10 s, retry.

RECORD_SHARED_WITH_DIFFERENT_SAE

ORIGIN_PROVISIONED

No shared keys were found for this pair.
Generate new keys now?
(skipped after a delete)

RMI deleteShare() on peer (handover):
peer removes its counterpart
on its row SAE first, then its own row

ORIGIN_PROVISIONED:
issuingSaeId == this SAE?

Shared keys already exist
with SAE … (created …).
Did you mean to use
those keys?

No

This will permanently delete
those keys. Do you really
want to delete them and
generate new ones?

Yes

generate again,
no prompt

Yes

Yes

No

Existing keys were kept.
No keys were sent to the device.

RMI synchronize() on peer SAE
(reversed hashes) → RECORD_SYNCHRONIZATION

getRecordMetadata() on peer.
This will permanently delete
those keys. Delete them and
generate new ones?

Shared keys already exist with
SAE … (created …).
Delete them and generate
new keys?

deliverExistingShare:
local row ORIGIN_PROVISIONED
and plainMax ≥ padPlainLen?

No

Origin SAE has not finished provision,
or the chip is smaller. No keys sent.

RECORD_SHARED_WITH_DIFFERENT_SAE

Peer outcome?

No

tryDelete local row.
Existing keys were kept.

other

Yes

No

TROPIC_WIPE_FINISHED

Yes

RMI getRecordMetadata()
on the origin (issuingSaeId)

unreachable

Origin SAE is unreachable.
Keys were not sent to the device.

INSERTED

lockFetch(): origin, then RMI on peer
→ RECORD_FETCHING_STARTED

lock refused

Rollback origin row. Target SAE
rejected the insert; retry later.

deleteShareAndRestart: RMI removeRecord()
on the row SAE first, then local
forceDelete; restart synchronize()

restart

OriginShareCheck.decide()

Yes

IN_PROGRESS: Origin SAE has not
finished provision. No keys sent.

IN_PROGRESS

DELETE_BOTH

DELETE_BOTH: RMI removeRecord() on
origin first, then local forceDelete.
Shared keys were deleted. Retry.

PROVISIONED

QkdPadMaterialFetcher.fetch()
enc_keys from the KME: key IDs + pads

RMI insert(record with key IDs)
peer: getKeyWithKeyIds at its KME,
HSM-wrapped pads, finishRecordInsert
→ RECORD_SHARED

sendStoredShare: RMI removeRecord()
on origin first, read the payload,
local forceDelete, remove payload

Every grey end closes TLS without a downlink. The chip has already erased its QKD R-MEM slots after the uplink. Exceptions (QKD, RMI) end the session with an operator error message.

Peer insert OK and origin
finishRecordInsert()
→ RECORD_SHARED?

Yes

No / exception

rollbackBoth: RMI removeRecord() on
peer first, then local tryDelete;
session ends with an error

SeKemFill.buildDownlink()
ML-KEM-768 encapsulation, sealed pads,
decrypt_half = ORIGIN

SeKemFill.buildDownlink()
ML-KEM-768 encapsulation, sealed pads,
decrypt_half = PEER

Wait for TROPIC_WIPE_FINISHED, then
stream the LV downlink v3
(device reports every 10 KiB)

secure_qkd_ingest()
stores all pads?
→ SE_OK

No

Abort / close TLS

Yes

Outcome Generated
(new origin share)?

Yes (Generated)

markOriginProvisioned(): RMI on peer
first, then local → ORIGIN_PROVISIONED
on both rows

No (Existing)

Both rows marked?

No

Device stored the keys, but the share was not
marked provisioned. Rows stay RECORD_SHARED.

Yes

Loading to device: 100 %
Share successful.